EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

A Resource-Aware ILP Solver for Low-Power Streaming CNNs on FPGA

by

Hussein, Y.

MSc Thesis

Thesis committee

dr. A. Naithani
dr.ir. M. D. Gomony
dr.ir. M. Geilen
ir. S. Oktem
dr. W. Yi

Graduation

Program:	Electrical Engineering
Research group:	Electronic Systems
Thesis supervisors:	dr. A. Naithani ir. S. Oktem
Date of defense:	17-11-2026
Student ID:	1316931
Study load (ECTS):	120

The research of this thesis has been carried out in collaboration with *Royal Philips*.
This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

A Resource-Aware ILP Solver for Low-Power Streaming CNNs on FPGA

Yaman Hussein, *Student, TU/e*

***Abstract*—FPGA streaming dataflow accelerators promise low-power and deterministic real-time inference for deep-learning models. However, their deployment success hinges on balancing task-specific accuracy and throughput against hardware resource utilization. Our work contributes a resource-aware integer linear program (ILP) optimizer, built on the FINN framework, that selects per-layer parallelism and quantization bitwidth to maximize performance within FPGA resource constraints. We also propose an efficient convolutional neural network (CNN) architecture via hardware-aware optimization, evaluated on semantic segmentation of X-ray coronary angiograms for the diagnosis of coronary artery disease—a use case for our deployment flow. The resulting optimized FPGA accelerator was evaluated against a CPU, a GPU and the Vitis AI DPU overlay on FPGA, and achieved a streaming throughput of 355 FPS at a Dice of 0.795 for 512×512 input images. Compared to an RTX 2050 GPU, its strongest contender, it used $5.5\times$ less total energy per image, at the cost of a higher latency of 6.87 ms. This work demonstrates how such a method can produce an energy-efficient solution for demanding segmentation tasks, and may extend to other deep-learning applications. For the proposed CNN, our ENet-based optimized architecture had over $14\times$ fewer parameters and FLOPs than the ENet baseline, at a cost of only about 2 Dice points on the ARCADE dataset.**

***Index Terms*—segmentation, ENet, CNN, FPGA, FINN, CAD, XCA.**

I. Introduction

DEEP learning is increasingly used in low-energy and high-performance edge applications. Autonomous vehicles, robotics, and handheld or wearable medical devices require real-time, high-throughput, or low-energy inference.

Computer vision is a common application of deep learning and is broadly categorized into image classification, object detection, and semantic segmentation. Indeed, it is one of the more computationally demanding forms of digital signal processing due to the large size and high dimensionality of images. Of the three tasks, semantic segmentation can be the most computationally demanding, as each input pixel is classified into one of several classes. This requires taking the neighborhood of each pixel into account using dense convolutions or matrix multiplications, further increasing the computational demand.

The execution of deep-learning models is known as inference, and is commonly done on graphics processing units (GPUs) or central processing units (CPUs). Field-programmable gate arrays (FPGAs) are an alternative that can be reconfigured at the hardware level. They support custom parallel digital circuits that implement deterministic, high-throughput, and energy-efficient solutions, which makes them suitable for streaming real-time applications. Deep-learning inference on FPGAs is broadly categorized into two paradigms. The first is the streaming dataflow model, where the algorithm is spatially unfolded into a pipeline of concurrent layers (circuits) on-chip. Data flows as a stream through these layers, hence the name. The second is the overlay accelerator, which commonly relies on a scheduler to compute the network layer by layer on a reusable architecture, loosely resembling execution on a GPU.

For real-time inference applications, dataflow architectures enable optimized solutions for higher energy efficiency, lower latency and higher performance. This, however, comes at the cost of a larger design space and greater design effort. Their execution also requires no scheduler, and hence is free of operating-system interference. Consequently, they offer reduced jitter, lower energy, and potentially more seamless integration with a broader processing system.

A major challenge of realizing a model as an FPGA dataflow accelerator is the complex design space exploration (DSE) it presents. The accelerator must achieve high performance in terms of task accuracy, throughput, and energy efficiency. It must balance this against using as few on-chip FPGA resources as possible, or allow trading them off to obtain a feasible system that can also host other processing tasks on-chip.

This is a multi-objective problem with many interdependencies. Achieving a higher accuracy at fixed throughput generally requires more parameters and floating-point operations (FLOPs) from a model and, in the case of FPGAs, a higher quantization bitwidth to increase the representational range. This in turn requires more computational parallelism at the expense of more hardware resources. To further complicate the DSE, various deep-learning architectures exist with no clear winner for a given deep-learning task or FPGA target. While frameworks such as FINN [1] and hls4ml [2] provide a toolchain for deploying such architectures on FPGAs, they do not provide a holistic DSE solution to explore the trade-offs between accuracy, throughput, and hardware resource usage. For the latter, none allows seamless trade-offs between look-up tables (LUTs), digital signal processing (DSP) slices and on-chip memory (OCM) such as block RAM (BRAM), LUT-based RAM (LUTRAM) or UltraRAM (URAM), which may limit feasibility under tight constraints. Consequently, we frame our work around two main research questions:

1) How does a streaming accelerator configured by an integer linear program (ILP) on the AMD XCZU7EV FPGA compare with CPU, GPU and Vitis AI deep learning processing unit (DPU) platforms in terms of energy efficiency, latency and throughput, and with FINN's automatic flow in terms of accuracy, feasibility, and resource efficiency?

2) By how much can hardware-aware ablation reduce the parameters and FLOPs of the ENet baseline for a segmentation task, and by how much does quantizing from 32-bit floating point (FP32) to 8-bit integers affect the accuracy on our dataset?

To anchor our effort, we rely on a challenging medical use case: the semantic segmentation of X-ray coronary angiography (XCA). It aims to separate the main coronary arteries from the noisy, low-contrast background of an X-ray image and to identify their type. This is eventually used to estimate the narrowing of their diameter, which may assist in the diagnosis of coronary artery disease (CAD), a leading cause of death worldwide [3], [4]. Notably, XCA has modest throughput and latency requirements in practice, but we use it to explore how far performance and energy efficiency can be pushed on this task, especially relative to other compute platforms.

Addressing our research questions, we contribute the following:

- An ILP-based optimization flow using the FINN framework, evaluated against a CPU, a GPU, and the Vitis AI DPU overlay. At equivalent Dice on the ARCADE dataset [5], it achieves the lowest energy per image of all platforms at real-time throughput. Compared to FINN's automatic flow, it uses fewer LUTs and is the only flow that fits the device. Under fixed resource caps, ILP-selected mixed precision either reaches a higher Dice or fits within the caps, compared to uniform-bitwidth quantization.
- A task- and hardware-aware optimization of a convolutional neural network (CNN) architecture, via a benchmarking study of prior-art models and a staged greedy ablation. The resulting model has over $14\times$ fewer parameters and FLOPs than ENet, at a cost of about 2 Dice points.

The rest of this paper is organized as follows. Section II introduces the dataset, training setup, evaluation metric, the ENet baseline architecture, the FINN dataflow framework and Hessian sensitivity. Section III reviews related work. Section IV describes the staged greedy ablation and the hardware deployment, including the ILP optimizer. Section V reports the ablation, benchmark, ILP and hardware results, and Section VI discusses and concludes our work.

II. Background

A. ARCADE Dataset

We use the ARCADE dataset [5], which contains two subsets of coronary angiography frames annotated by experts. The first subset labels 26 artery segments defined by the SYNTAX score [6], while the second labels regions with stenosis. Our work uses the former, containing 512×512 grayscale frames split into 1200 labeled images for training and validation and 300 test images for evaluating the models. The 26 SYNTAX segment labels are grouped into the four main coronary branches, namely the left anterior descending artery (LAD), right coronary artery (RCA), left circumflex artery (LCX) and left main (LM) artery, as exemplified in Fig. 1. This allows stenosis to be localized downstream for each main branch using its diameter profile [4], [6]. The reduction in the number of classes may also reduce class-border ambiguity in the clinical annotations, which can improve model accuracy. It may also reduce the number of parameters and FLOPs needed, allowing for more efficient deployment.

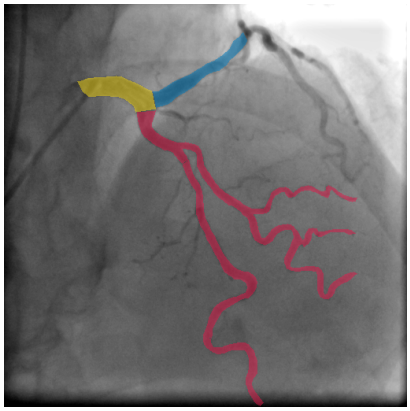


Fig. 1. Example ARCADE frame with its branch labels: LAD (red), LCX (blue), LM (yellow). RCA is not visible in this view.

B. Training Setup

All models are trained inside the nnU-Net v2 framework [7], which handles preprocessing, data loading, augmentation, the training loop and checkpoint selection. Using a standardized training pipeline keeps the comparison fair across the architectural ablation and all result evaluations. We use the 2D configuration with its standard on-the-fly augmentation and per-image z-score normalization for preprocessing. Training minimizes the sum of the cross-entropy and soft Dice losses using AdamW (initial learning rate 10^{-3}, weight decay 10^{-2}) with a polynomial learning-rate decay. Each model is trained for 150 epochs of 250 iterations at batch size 12 on full 512×512 images, using 1000 training and 200 validation images, and the checkpoint with the best validation Dice is kept.

C. Accuracy Metric

Class overlap is measured with the Dice similarity coefficient (hereafter Dice), a standard metric in image segmentation that quantifies the overlap between a predicted mask and the ground truth. It is the harmonic mean of precision and recall and ranges from 0 (no overlap) to 1 (perfect overlap). For a single test case and a single class c, let G_c and P_c be the sets of pixels labeled c in the ground truth and in the prediction, respectively. Dice is the ratio of twice the overlap to the total size of both masks:

$$\mathrm{Dice}_c = \frac{2\,|G_c \cap P_c|}{|G_c| + |P_c|} = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad (1)$$

where TP_c, FP_c and FN_c are the numbers of true-positive, false-positive and false-negative pixels for class c. By convention, $\mathrm{Dice}_c = 1$ when both G_c and P_c are empty, where the ratio is undefined. Otherwise, a constant of 10^{-8} is added to the denominator in our implementation, which changes the result negligibly.

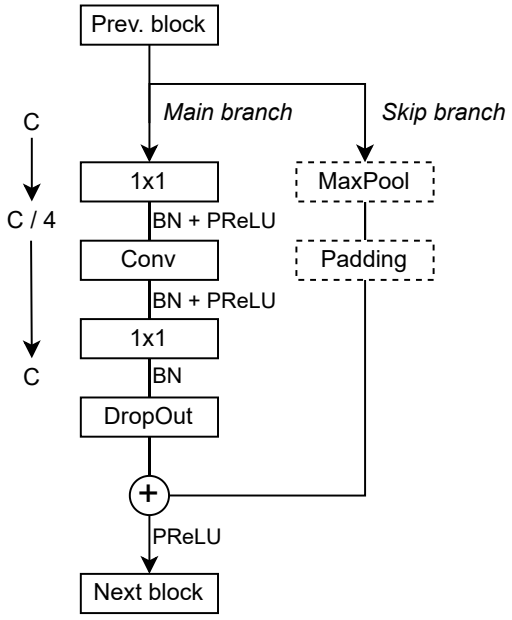


Fig. 2. ENet regular bottleneck. The dashed MaxPool and Padding on the skip path are only present in downsampling bottlenecks to reduce spatial size.

Each class score is the mean of the per-case scores over the N test cases. The reported Dice is the equally weighted mean of the class scores over the $K = 4$ foreground classes, not weighted by pixel count or by case.

D. ENet

ENet [8] is a lightweight encoder-decoder segmentation network with 0.37 M parameters, designed for real-time use. It downsamples early to shrink feature maps and is built from residual bottlenecks (Fig. 2).

A bottleneck adds the outputs of a main branch and a skip branch. The main branch applies a pointwise 1×1 projection to fewer channels, an inner convolution, and a 1×1 expansion back, with batch normalization and parametric rectified linear unit (PReLU) activations. The inner convolution is a regular or dilated 3×3 convolution, an asymmetric 1×5 followed by 5×1 convolution, or, in upsampling bottlenecks, a transposed convolution. The skip branch is the identity, except in downsampling bottlenecks, where max-pooling stores the indices of the maxima, and in upsampling bottlenecks, where max-unpooling uses these indices to restore the spatial size.

The network consists of an initial block, encoder stages that begin with a downsampling bottleneck, and decoder stages that begin with an upsampling bottleneck, with widths of 16, 64, 128, 64 and 16 channels in stages 0 to 4 (stage numbering as in Table I, which lists our design at a quarter of these widths). These widths are the baseline reference widths in our work, later reduced by fixed reduction factors. Furthermore, the deepest and longest stage, named the context stage, alternates regular, dilated and asymmetric bottlenecks.

E. FINN Framework

FINN [1], [9] is an open-source compiler by AMD that maps a mixed-precision neural network onto an FPGA as a standalone streaming dataflow accelerator. FINN unfolds all layers into dedicated processing engines, interconnected with first-in-first-out (FIFO) buffers. This means all weights and intermediate activations may remain on-chip.

Streaming flows differ from their overlay counterparts, such as the Vitis AI DPU [10], where a fixed, prebuilt 8-bit quantized processor executes the network layer by layer (time-multiplexed) from external memory, and relies on CPU control to schedule the execution.

In this framework, the convolutions are lowered to matrix-matrix products. A sliding-window unit (SWU) generates the input vectors, and a matrix-vector-activation unit (MVAU) multiplies them with the weight matrix. The MVAU consists of P parallel processing elements (PEs), each with Q single instruction, multiple data (SIMD) input lanes. The numbers P and Q are configurable parameters that determine the temporal folding, and thus the computational parallelism per layer. The MVAU also supports arbitrary bitwidths for weights (W) and activations (A), denoted b_W and b_A, which are also configurable parameters that dictate the precision of the computation. At the expense of more on-chip resource usage, increasing P and Q of a layer may increase the accelerator's throughput, while increasing b_W and b_A may increase its inference accuracy.

FINN absorbs batch normalization and quantized activations into multi-level threshold units, each with $2^b - 1$ comparison thresholds. Pooling, residual add, stream duplication, concatenation, padding, upsampling and argmax are separate prebuilt streaming engines. All of these are interconnected with FIFOs and data-width converters to handle the arbitrary bitwidths.

Throughout this paper, throughput means streaming throughput: the sustained frame rate at batch size 1, as required by real-time streaming applications, where batching would add latency. FINN's native solver takes a target throughput in frames per second (FPS) and greedily sets P and Q to balance the number of cycles per frame across all layers. Furthermore, the interconnecting FIFOs are autosized via a hardware simulation to prevent stalls and performance degradation. The solver does not set bitwidths, and it does not consider the resource usage of the whole network under the resource constraints of the target FPGA.

F. Hessian Sensitivity

Our optimizer uses Hessian sensitivity to allocate a bitwidth to each layer. It estimates how much the loss increases when a layer is quantized to a given bitwidth, by combining the loss curvature around the trained weights and activations with the quantization error they would incur. A larger increase typically means lower accuracy, so it serves as a proxy for post-quantization accuracy. It is computed for each layer and bitwidth pair and normalized for use in the ILP (Section IV-B5).

Following HAWQ-V2 [11], the loss change from quantizing a tensor θ to b bits is approximated as

$$\Omega^{(b)} = \frac{\mathrm{Tr}(H)}{n} \left\| Q_b(\theta) - \theta \right\|_2^2, \tag{2}$$

where θ is the weight or activation tensor of a layer with n elements, H is the Hessian of the loss with respect to θ, and $Q_b(\theta)$ is θ fake-quantized to b bits. A layer is thus costly to quantize if the loss is sharp around it (large mean curvature $\mathrm{Tr}(H)/n$) or if the bitwidth is low enough to cause a large quantization error.

Equation (2) follows from a second-order expansion of the loss around the FP32 checkpoint. It assumes that the checkpoint is close to a loss minimum, that the quantization error is small and spread evenly over the eigendirections of H, and that the errors of different layers are uncorrelated, so that cross-layer terms vanish and the layer scores are additive. As in HAWQ-V3 [12], the last assumption lets the ILP minimize the sum of the layer scores, but it does not hold in practice, especially for activations, which are interdependent downstream.

We estimate $\mathrm{Tr}(H)$ per image with Hutchinson's method, using 40 Rademacher probes and Hessian-vector products from double backpropagation. The trace is evaluated with the training loss on the FP32 network over all $N = 1200$ training and validation images. For weights, θ is the layer's weight tensor, and for activations it is the layer's output, captured with a forward hook. Both traces are divided by the element count and averaged over images. Q_b is symmetric per-tensor uniform quantization with scale $\max|\theta|/(2^{b-1}-1)$.

III. Related Work

A. Lightweight CNNs for Real-Time Inference

Efficient CNNs have been proposed for real-time inference. ENet [8] (Section II-D) is a lightweight residual encoder-decoder network with only 0.37 M parameters. ERFNet [13] follows the overall structure of ENet, but uses non-bottleneck residual blocks with factorized dilated convolutions in its deepest encoder stages, and improves the mean intersection over union (mIoU) over ENet by about 10 points on Cityscapes. Beyond these, we benchmark nine further models on ARCADE (Table IV), including the large nnU-Net and ResNet101-UNet baselines. Most of the others have fewer parameters and FLOPs, simpler and shallower dataflow graphs, and less complex operations, which makes them favorable for FPGA streaming dataflow deployment.

B. CNNs on FPGA

ENet-based models have been deployed on time-multiplexed FPGA overlay accelerators, but reach only 19.5–130.75 FPS [14]. Other works deploy ENet variants as streaming dataflow architectures. P-ENet [15] is a uniformly 4-bit quantized variant deployed on an AMD XCZU19EG FPGA using FINN. It reports up to 226 FPS at 70.33% mIoU on Cityscapes, and 1.44× less power consumption than a Jetson GPU, without stating the image size. ENetHQ [16] is deployed on an AMD XCZU9EG using hls4ml, with mixed-precision quantization and filter-ablation pruning, and reports 4.9 ms latency per 240×152 image at 36.8% mIoU on Cityscapes. None of these proposes a reusable, mixed-precision, resource-aware solver with analytically sized FIFOs, or targets the XCA segmentation task.

C. Dataflow DSE on FPGA

Optimizing FPGA dataflow inference throughput, accuracy and resource usage is a known challenge in FPGA-based deep learning. For throughput, greedy solvers include FINN's solver [1], [9], which sets P and Q to balance the number of cycles per frame, but does not take target device resource constraints into account. nn2fpga [17] uses an enumerated binary ILP to optimize the throughput under DSP and OCM constraints, but it neither determines activation FIFO buffer sizes nor supports non-power-of-two bitwidths or the dilated convolutions used by ENet.

For quantization accuracy optimization, ENetHQ [16] selects per-layer quantizers and filter counts with AutoQKeras, which requires retraining at each step and is time-consuming. HAWQ-V3 [12] uses an ILP to select bitwidths from Hessian sensitivity under model size, bit-operation and latency constraints, but it targets GPU deployment and is not aware of FPGA resource usage or dataflow throughput.

To the best of our knowledge, no prior work proposes an ENet-based CNN with a dilated-only context stage, nor jointly optimizes bitwidths, dataflow folding and analytically sized FIFOs in a single resource-aware ILP for FINN streaming dataflow accelerators.

IV. Method

This section describes the methodology of our two interconnected contributions, whose end-to-end flow is shown in Fig. 3. Model benchmarking takes prior-art models as input and selects a starting model, which the staged greedy ablation then compresses into an optimized model (Section IV-A). The ILP optimizer takes this model together with the objective and resource constraints, and outputs the FIFO depths, per-layer bitwidths and P/Q folding (Section IV-B1), and, after quantization-aware training (QAT) fine-tuning, the quantized weights. FINN takes these as input and outputs stitched Vivado intellectual property (IP) blocks, which are built into the bitstream for deployment on the FPGA (Section IV-B8).

A. Staged Greedy Ablation

Prior-art models are benchmarked on our task to determine a suitable starting architecture for subsequent compression. All benchmarked models and all ablation variants share the same training budget, initialization and evaluation procedure: the nnU-Net pipeline of Section II-B, random initialization from a fixed seed, and evaluation on the same held-out test set. The only exception is the ImageNet-pretrained ResNet101-UNet (Table IV). Various vision models, including convolutional, self-attention and state-space model architectures, are evaluated on their segmentation accuracy, parameter count, FLOPs, and hardware suitability.

From these, ENet is selected due to its lightweight, structurally simpler architecture and more standard operations than the rest. It further has the fewest parameters of the prior-art models (0.37 M) and the highest Dice among those below 5 G FLOPs (Table IV). MobileNetV3-Small is a comparable alternative, with fewer FLOPs (0.72 G) but more parameters (0.51 M) and a lower Dice (0.7937). We select ENet but

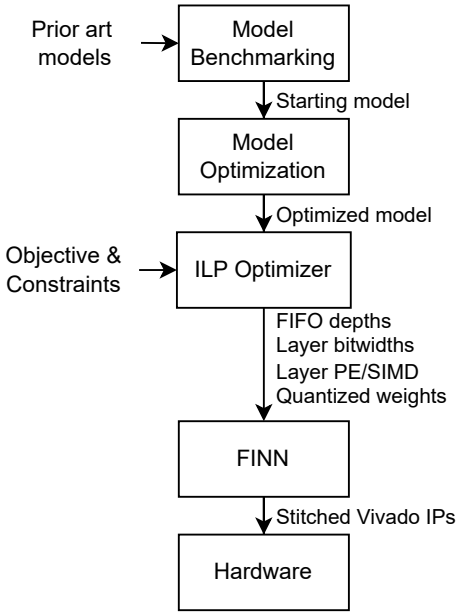


Fig. 3. Overview of the method. Each stage passes its result to the next, and the target objective and constraints are supplied at the ILP stage.

acknowledge MobileNetV3-Small as a strong candidate for future work. Next, a staged greedy ablation is conducted to optimize four separate axes: Dice, parameter count, activation element count, and FLOPs.

Dice is a proxy for clinical usefulness, while the other three metrics are proxies for hardware dataflow resource costs. The parameter count represents on-chip weight memory (BRAM[1], LUTRAM or URAM), and the FLOPs represent computation logic (LUTs and DSPs). The activation element count is a proxy for the OCM cost of the intermediate activations, which remain in OCM buffers. It counts the elements in the sliding-window buffer that each streaming convolution or max-pooling layer needs to sustain its operation, i.e., buffered pixels $\times$ channels. Summing this over all such layers of a network gives the *activation element count*, where the buffer of a layer is given by:

$$B_\ell = \begin{cases} 0, & k_h = k_w = 1 \\ d_h\,(k_h - 1)\,W_\ell\,C_\ell, & k_h > 1 \\ d_w\,(k_w - 1)\,C_\ell, & k_h = 1,\ k_w > 1 \end{cases} \tag{3}$$

Here k_h, k_w are the kernel height and width, d_h, d_w the dilation factors, and W_ℓ and C_ℓ the width and input channel count of layer ℓ's input feature map. The count is in elements, not bits or memory blocks, and excludes stride, padding, FIFOs (priced only in the ILP, Section IV-B1) and upsampling layers, so it does not distinguish between upsampling methods (Table II).

For each stage of the ablation, we ablated one or more architectural choices, retrained the resulting models, and evaluated them on ARCADE to reveal their impact on the four axes. At each stage, the best-performing model was carried over to the next. The final model is presented in Section V-A and compared against the ENet baseline as well as the other prior-art models.

1) Whole Network: Network-wide changes included dividing all ENet stage channels (baseline channels in Section II-D) by a power of two (1, 2, 4, 8, 16) to compress the model at a suitable accuracy-size trade-off, a form of structured pruning [16]. Unlike post-training pruning, each reduced-width model is trained from scratch. The activation function (PReLU or rectified linear unit (ReLU)) was also varied.

2) Context Stage: Next, the residual bottlenecks of the context stage, i.e., the deepest and longest stage (Section II-D), were varied. The ablations cover the inner convolution type (dilated or asymmetric) and the stacking pattern of these bottlenecks, which diverges from the ENet pattern of regular, dilated, asymmetric, dilated, and so on.

3) Decoder: Finally, we ablated the decoder upsampling method, since ENet's max-unpooling needs the pooling indices forwarded from the matching downsampling stage, and storing them is more complex in hardware because it requires memory management. We therefore tested simpler upsampling methods such as nearest-neighbor and bilinear interpolation.

B. Hardware Deployment

1) ILP Optimizer: After selection of the final architecture, the trained model is prepared for FPGA deployment via FINN. This involves quantization by choosing each layer's weight and activation bitwidths (b_W, b_A), determining its folding (P, Q), and selecting the FIFO depths between layers. Post-training pruning is possible but is not considered in this work and is left as future work.

Given the large combinatorial design space, an ILP is used as a DSE tool. Following the formulation of HAWQ-V3 [12], it selects the bitwidths via a Hessian sensitivity proxy, and decides the folding and FIFO depths of every node subject to LUT, BRAM and DSP budgets, based on FINN's analytical estimation model. Our ILP has two modes defining the optimization objectives, namely maximum FPS or maximum accuracy, depending on the goal. The objectives are kept separate so that each solve is single-objective and targeted at one goal, rather than tracing a Pareto frontier. Combining them lexicographically, with a user-specified tolerance ε as in (13), is left as future work.

The remainder of this subsection covers the optimizer's implementation and the deployment flow.

2) Design Choices: While FINN supports URAM in some cases, we intentionally avoid it: it cannot be initialized as ROM for weight and threshold storage, and its fixed 4096×72 bit aspect ratio would leave it underutilized for FIFO buffers, for which BRAMs are more efficient.

Also, while FINN supports fusing thresholding into the MVAU, doing so prevents the selection of a register-transfer level (RTL) MVAU capable of efficient DSP packing, and would therefore underutilize our board, on which DSPs are abundant.

Furthermore, because the standard FINN flow, which determines FIFO depths empirically via hardware simulation,

[1]Throughout this paper, BRAM refers to the BRAM36K block, a 36 Kb (36 864-bit) block RAM that can be split into two independent 18 Kb blocks.

severely overestimated the required depths for our network, we model the FIFO depths analytically first, followed by a controlled tuning simulation (Section IV-B7).

3) Variables and Constraints: A node $n \in \mathcal{N}$ is a computational block in the dataflow graph, and a network layer is made up of several nodes (e.g., SWU, MVAU, thresholding, pooling). The MVAU is the only node with weights, so it is the only node with a bitwidth choice. Let $\mathcal{I} \subseteq \mathcal{N}$ be the set of MVAU nodes, indexed by ℓ. Both modes share the following constraints:

$$y_{\ell,b},\ z_{n,f,b} \in \{0,1\} \tag{4}$$
$$\textstyle\sum_b y_{\ell,b} = 1 \qquad \forall \ell \in \mathcal{I} \tag{5}$$
$$\textstyle\sum_f z_{\ell,f,b} = y_{\ell,b} \qquad \forall \ell \in \mathcal{I},\ b \tag{6}$$
$$\textstyle\sum_{f,b} z_{n,f,b} = 1 \qquad \forall n \in \mathcal{N} \tag{7}$$
$$b_A(n) = \max_{p \in \mathrm{src}(n)} b_A(p) \qquad \forall n \in \mathcal{N} \setminus \mathcal{I} \tag{8}$$

$$\textstyle\sum_{n,f,b} z_{n,f,b} R^r_{n,f,b} + \mathrm{FIFO}^r \le \rho_r R^r_{\mathrm{total}}, \quad \forall r \tag{9}$$
$$C_n = \textstyle\sum_{f,b} z_{n,f,b}\, c_{n,f,b} \le f_{\mathrm{clk}}/\mathrm{FPS}_{\mathrm{target}}, \quad \forall n \in \mathcal{N} \tag{10}$$

Equation (4) defines the binary decision variables: $y_{\ell,b} = 1$ if MVAU $\ell \in \mathcal{I}$ uses the bit pair $b = (b_W, b_A) \in \{4,6,8\}^2$, and $z_{n,f,b} = 1$ if node $n \in \mathcal{N}$ uses fold $f = (P,Q)$ at bits b. Equations (5) and (6) select one bit pair per MVAU and tie its fold to it, and (7) selects one fold for every node. In (8), nodes without weights, such as thresholding, inherit the activation bitwidth b_A of their source MVAUs $\mathrm{src}(n) \subseteq \mathcal{I}$ (both branches for a residual join), so that every node can be priced by the cost model; the maximum is encoded exactly with linear constraints. The resource usages $R^r_{n,f,b}$ and cycle counts $c_{n,f,b}$ come from the calibrated cost model (Section IV-B6) and are enumerated offline for every node, fold and bit pair, so the model stays linear in y and z. In (9), $r \in \{\mathrm{LUT}, \mathrm{BRAM}, \mathrm{DSP}\}$, FIFO^r is the analytically estimated FIFO usage, and $\rho_r \in (0,1]$ is the fraction of the device capacity R^r_{total} made available to the optimizer. Choosing $\rho_r < 1$ leaves headroom for the cost model's estimation error (Table V) and for place-and-route. In (10), C_n is the cycles per frame of node n, which the optional throughput target bounds by the clock budget $f_{\mathrm{clk}}/\mathrm{FPS}_{\mathrm{target}}$. In the maximum FPS mode, the bits are fixed and the index b is dropped.

4) Maximum FPS Mode: This mode is used to achieve the highest throughput deemed feasible by our estimation model, given the resource constraints. The bitwidths are fixed, so only the folds $z_{n,f}$ of all nodes are optimized. Two ILPs are solved in sequence, both subject to (4)–(10), and linked by T^*:

$$\mathcal{R} = \tfrac{1}{3} \textstyle\sum_r \left(\sum_{n,f} z_{n,f} R^r_{n,f} + \mathrm{FIFO}^r \right) / R^r_{\mathrm{total}} \tag{11}$$
$$T^* = \min_{z,\,T} T \quad \text{s.t.} \quad T \ge C_n \ \ \forall n \tag{12}$$
$$\min_{z} \mathcal{R} \quad \text{s.t.} \quad T \le (1+\varepsilon)\, T^* \tag{13}$$

Equation (11) defines the mean normalized resource use $\mathcal{R}$ over LUT, BRAM and DSP, including FIFOs. The first ILP minimizes the cycles per frame T of the slowest node, which maximizes the throughput f_{clk}/T^*. The second minimizes $\mathcal{R}$ while keeping T within a tolerance ε of T^*, which avoids infeasibility.

5) Maximum Accuracy Mode: This mode is used to achieve the highest post-quantization accuracy deemed feasible by our estimation model, given the resource constraints and FPS target. The weight and activation bitwidths b_W and b_A of each MVAU are searched separately, and concurrently with the folds. For each MVAU $\ell \in \mathcal{I}$, a binary variable $y_{\ell,b}$ selects a bit pair, i.e., $b = (b_W, b_A) \in \{4,6,8\}^2$, and each pair carries a normalized sensitivity $\hat{s}_{\ell,b} \in [0,1]$ as the accuracy proxy. These sensitivities are precomputed once on the trained FP32 checkpoint, before the solve, and stored as binary files, so the ILP only looks up the values for every layer and bit pair and selects among them, without evaluating the network itself.

Two ILPs are solved in sequence, both subject to (4)–(10), and linked by S^*:

$$\bar{S} = \tfrac{1}{|\mathcal{I}|} \textstyle\sum_{\ell \in \mathcal{I}} \sum_b \hat{s}_{\ell,b}\, y_{\ell,b} \tag{14}$$
$$S^* = \min_{y,z} \bar{S} \tag{15}$$
$$\min_{y,z} \mathcal{R} \quad \text{s.t.} \quad \bar{S} \le (1+\varepsilon)\, S^* \tag{16}$$

Equation (14) defines the mean sensitivity $\bar{S}$ of the selected bit pairs, the accuracy proxy. As in the maximum FPS mode, the first ILP minimizes $\bar{S}$, and the second minimizes $\mathcal{R}$ from (11) while keeping $\bar{S}$ within a tolerance ε of S^*.

Our setup uses the Hessian sensitivities of HAWQ-V2 and the ILP formulation of HAWQ-V3 (Section II-F), with the quantization error computed by Brevitas fake-quantization. The sensitivities are min-max normalized once, jointly over all layers and bit pairs rather than per layer, so the relative magnitudes between layers, which already carry the Hessian trace, are preserved. The layers are combined by an unweighted mean, which inherits the additivity limitation discussed in Section II-F.

6) Analytical Model Calibration: The ILP prices every node with closed-form formulae for LUT, BRAM, DSP and cycle counts as functions of the node's bitwidths (b_W, b_A) and fold (P,Q). They start from the FINN and FINN-R estimation models, which the authors acknowledge can be inaccurate, with LUT counts reported to be off by up to 30% [9], which would require an iterative solve and synthesis loop to find a feasible design.

Therefore, to improve its accuracy, we calibrated the analytical model by fitting the formulae to post-route per-node utilization from out-of-context syntheses of previous builds. Added constants were fitted per node type by least squares on a dataset from multiple builds. The fits depend on P and Q, and on bitwidth only where it matters (thresholds or width converters). The fits cover our chosen model architecture and design choices, and may not generalize to other architectures.

7) FIFO Tuning: Once the ILP has selected a configuration, a lightweight dataflow simulator is used to tune the FIFO depths between layers. Since the ILP's analytical model accounts for neither the interaction between layers nor the dataflow graph, the simulator adjusts the FIFO depths to resolve any deadlocks and minimize performance loss due to burstiness, and/or reduce them to avoid overallocation. This

simulation is neither cycle-accurate nor hardware-faithful, but it is fast and sufficient to detect deadlocks and performance loss.

First, the residual blocks are simulated independently. Here, the analytically chosen FIFO depths are relaxed, the performance of the bottleneck is evaluated, and the FIFOs are subsequently shrunk to their measured maximum occupancy. Next, blocks are simulated in adjacent pairs to tune their interconnecting FIFOs. Finally, the full network chain is simulated to warn early in the flow of unresolved deadlocks or missed performance targets. As the simulator is not hardware-faithful, its result is validated downstream: after FINN generates and interconnects the IP blocks, an RTL simulation of the design confirms the absence of deadlocks and performance degradation (Section V-B3), and the deployed design's throughput is measured on the FPGA.

FINN's flow, in contrast, sizes each FIFO to its maximum observed occupancy. While this may prevent any stall at all, it can use excess OCM, which inflates the FIFO sizes for our architecture (Section V-B3). Our simulator instead feeds the network with a paced source and sizes the FIFOs large enough to sustain the target throughput, and accepts stalls beyond it.

8) Deployment on FPGA: For every quantized model, once the final network is mapped to a FINN-compatible graph, the quantizer scales are calibrated on the training images, and the network is then fine-tuned with QAT using Brevitas for 15 epochs under the same training conditions as in FP32 training. This recovers accuracy lost to quantization error. The QAT recipe is untuned and is left as future work.

Finally, the model graph with the quantized weights, folding, and FIFO sizes is given as input to our FINN build script, which includes custom graph transformations to make the network hardware-compatible and generates the FPGA bitstream. The accelerator is then deployed and evaluated on the target FPGA for latency, throughput, and resource utilization.

V. Results and Discussion

A. Staged Greedy Ablation Results

Following the staged greedy ablation (Section IV-A), Table I lists the resulting architecture, which is expressed relative to ENet but diverges from it in several ways:

- A quarter of the channels per stage compared to ENet, the smallest width before a steep Dice drop. Relative to the full-width design, it removes 93% of the parameters and FLOPs and 75% of the activation elements, at a cost of 4.03 Dice points (Table II).
- Bilinear interpolation for upsampling instead of max-unpooling, which needs no stored pooling indices and improves Dice by 13% at this width.
- A custom context stage with only dilated convolutions, stacked in repeating sequences of dilations 2, 4, 8, 16, as it was the most accurate context stage. With about the same parameters and FLOPs, it gains 4.05 Dice points over the ENet context stage, at the cost of 54% more activation elements. We use twice as many dilated convolutions as ENet, and no regular or asymmetric convolutions.
- ReLU activations instead of PReLU, since ReLU maps to a single FINN threshold unit, at a small cost in FP32 Dice (discussed below).

TABLE I
Chosen model architecture relative to ENet. Dilation arrows in stage 2 denote the order of dilations in the stacked blocks.

	Stage	Bottleneck type	No. blocks	No. channels	Output size
Encoder	0	Initial	1	4	256×256
	1	Downsampling	1	16	128×128
		Regular	4		
	2	Downsampling	1	32	64×64
		Dilated (2→4→8→16)	16		
Decoder	3	Upsampling	1	16	128×128
		Regular	2		
	4	Upsampling	1	4	256×256
		Regular	1		
	5	Transposed conv.	1	4	512×512

Table IV compares our models with the ENet baseline and the other prior-art models benchmarked on ARCADE. All models are trained from scratch under the identical conditions described in Section IV-A, except ResNet101-UNet, whose encoder is initialized with ImageNet-pretrained weights and then trained on ARCADE like the others. Ours-L is the full-width variant of our design and Ours-S the $\times 1/4$ variant deployed on the FPGA. Ours-L improves on ENet by about 2 Dice points with slightly fewer parameters and FLOPs, and comes within 1 point of ERFNet at about 6–7$\times$ fewer parameters and FLOPs. Ours-S is the smallest and cheapest model, with over $14\times$ fewer parameters and FLOPs than ENet at a mean Dice of 0.7876, less than 1 point below MobileNetV3-Small despite $21\times$ fewer parameters.

Table II ablates our final model one design choice at a time. The "no dilated" and "no asymmetric" variants replace the respective convolutions in the ENet context block with regular 3×3 ones. Each variant is trained from scratch using our standardized training setup (Section II-B).

Each halving of the channel width removes a similar share of the parameters (69–74%), yet the Dice loss grows from 1–3 points over the first two halvings to 9 and 26 points over the next two. This accelerating loss is consistent with model capacity becoming the bottleneck below $\times 1/4$, so we take $\times 1/4$ as the smallest width before the steep drop. We do not choose $\times 1/2$ because its small gain over $\times 1/4$ is outweighed by its cost: our cost model predicts that, at 8 bits and a target of 150 FPS, it requires 206.9k LUTs (90% of the AMD XCZU7EV on the ZCU106 board) instead of 147.3k (64%), and 235.5 BRAMs (75%) instead of 102 (33%). At such utilization, routing and timing closure become difficult, and since the calibrated cost model tends to underestimate LUTs (Table V), the build would likely be infeasible.

To understand why max-unpooling underperforms, we applied several ablations to it at multiple widths (Fig. 4). We (i) zeroed the output of the unpooling skip branch to understand its contribution, (ii) replaced the pooling indices with random ones (15 runs, mean and range shown) to verify the signal from the pooling indices, (iii) filled all four pixels of each

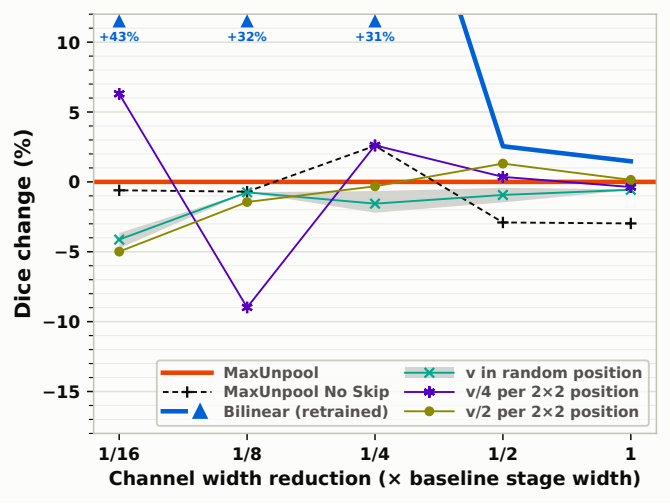


Fig. 4. Relative Dice change (%) of each post-training ablation with respect to the trained max-unpooling model of the same width. Bilinear is the only retrained network shown.

2×2 window with $v/4$ or $v/2$, where v is the pooled value, and (iv) replaced max-unpooling with bilinear upsampling, and retrained from scratch as a final comparison. All but the bilinear variant are inference-time substitutions without retraining. The $v/4$ fill matches the total weight (sum) of the max-unpooling's sparse output, and $v/2$ matches its L2 norm (energy), so that the skip keeps its magnitude when added to the main branch.

Max-unpooling is as good as or better than the zeroing, random-index and dense-fill ablations at full width, but is occasionally worse at smaller widths. At $\times 1/4$, zeroing the skip branch even improves Dice by about 2.6%, which suggests that the unpooled signal is weak or confusing to the model. Random indices reduce Dice only slightly, which suggests that the index positions matter little. The dense $v/4$ and $v/2$ fills perform comparably to max-unpooling from $\times 1/4$ upwards (except $v/4$ at $\times 1/8$), which suggests that the density (weight) of the upsampled signal matters more than its position. Bilinear upsampling (with PReLU activation) is a smooth interpolating version of these dense fills, and after retraining, it outperforms max-unpooling at all widths, by about 1.5% at full width and by more than 30% at $\times 1/4$ and below, with a gap that grows as the width shrinks.

Our dilation-heavy model is motivated by comparing the ENet context stage with its no-dilation counterpart in Table II, where Dice drops from 0.75 to 0.40. The binary vessel Dice, however, drops only from 0.72 to 0.66, so the loss is mostly in identifying which artery a vessel belongs to, and not in finding the vessel at all. Of the detected vessel pixels, the share assigned to the correct artery falls from 93.9% to 73.9%. We attribute this to vessels looking alike locally: a short segment of the LAD, LCX or RCA has a similar width and appearance, so its identity depends on where it lies in the wider vessel tree. Dilated convolutions enlarge the receptive field without extra parameters, which could explain the improvement in class identification: detecting a vessel relies mostly on local appearance, whereas identifying the artery needs wider context, so the class-wise Dice improves more than the binary Dice.

Compared to ReLU, PReLU improves Dice by 1.3% in FP32 (0.7876 to 0.7979), but changes it by -9.2% at 4 bits and by only $+0.2\%$ to $+2.8\%$ at 6–8 bits. We attribute the degradation at low bitwidths to the signedness requirement of PReLU, which costs one bit of positive-side resolution. While PReLU shows promise, its occasionally negative slope makes it a non-monotonic (V-shaped) function, which cannot be represented by a single threshold unit in FINN. It must instead be built as a step-wise function from two parallel thresholding units in a fork/add configuration. Our cost model predicts that, even after the additional LUT cost of this construction, a 6-bit PReLU build saves only 4.9% of the LUTs of the XCZU7EV compared to an 8-bit ReLU build of the same accuracy, which is well within the LUT estimation error of the cost model (Table V). Therefore, we do not implement it and leave it as future work.

B. ILP Results

1) Maximum Accuracy Mode Results: To demonstrate the ILP's ability to find a feasible solution given constraints, we ran the ILP optimizer with a target throughput of 150 FPS and resource caps of 40% of the LUTs, BRAMs and DSPs available on the XCZU7EV, for four different builds. The first three had the bitwidths fixed at uniform 4, 6 and 8 bits for both weights and activations, leaving the folding factors and FIFO depths as the only free variables. The fourth build was solved under the maximum accuracy objective, in which the bitwidths (4, 6 or 8 bits) of each layer's weights and activations, as well as its folding, were optimized. Unlike its 4- and 6-bit counterparts, the uniform 8-bit build was infeasible under the constraints, so it was re-solved with higher caps for comparison purposes. The mixed build had an average weight and activation bitwidth of 7.86 and 6.91 (combined average of 7.39) bits, respectively, and was feasible under the same constraints.[2] Fig. 5 shows the estimated resource consumption of the four builds against their Dice, after fine-tuning with QAT to recover the accuracy lost due to quantization.

As expected, increasing the quantization bits increases the final Dice, but also the resource consumption, as wider operands need larger multiplier logic (LUT, DSP) and more buffering and weight storage in OCM. The 6-bit build uses an estimated 39% of the LUTs and the 8-bit build 63%, beyond the 40% cap. The mixed build reaches a Dice of 0.7843 against 0.7712 for the 6-bit build (+1.3 points) at about the same LUT share (40% against 39%), by spending the remaining budget unevenly: it uses more DSPs (15% against 8.5%) and BRAM (40% against 31%), reaching the cap while remaining feasible. Over three QAT seeds per build, the mixed build (0.7869 ± 0.0043 Dice) outperforms the uniform 6-bit build (0.7736 ± 0.0020) by 0.013, more than three times the standard deviation of either build, so the gap is not explained by seed variability.

[2]Feasibility here refers to the analytical estimates meeting the 40% cap, which is well below the device capacity and leaves headroom for the estimation error (Table V). All builds were implemented in FINN and shown to be feasible on FPGA.

TABLE II
ABLATION OF OUR FINAL MODEL (OURS-S), CHANGING ONE SETTING PER ROW. ENET IS SHOWN FOR COMPARISON. BOLD: BEST PER COLUMN.

Variation	Settings	Params	Act. elements	FLOPs (M)	Dice
Ours-S	ReLU, bilinear, $\times 1/4$ width, dilation-only context	24 261	139 264	297.3	0.7876
Activation	PReLU	25 201	139 264	297.3	0.7979
Upsampling	Nearest neighbor	24 261	139 264	297.3	0.7734
	Max unpooling	24 261	139 264	297.3	0.6947
Channel width	$\times 1$	361 452	552 448	4214.1	**0.8279**
	$\times 1/2$	92 592	276 992	1099.3	0.8165
	$\times 1/8$	6 745	71 680	111.7	0.6995
	$\times 1/16$	**2 088**	37 888	**64.5**	0.4365
Context stage	ENet context	24 581	90 240	299.4	0.7471
	ENet context, no dilated	24 581	**36 992**	299.4	0.3986
	ENet context, no asymmetric	24 261	86 016	297.3	0.7550
ENet [8]	PReLU, max unpooling, $\times 1$ width, ENet context	369 692	356 352	4247.7	0.8091

TABLE III
HARDWARE BENCHMARK OF THE 8-BIT NETWORK (512×512 300 TEST IMAGES, EXCLUDING DATA LOADING AND PREPROCESSING).

Platform	Method	Max clock (MHz)	Latency (ms)	Streaming FPS	Total energy (mJ/img)	Dynamic energy (mJ/img)
GPU (RTX 2050)	TensorRT	2100	3.05 ± 0.19	281.7 ± 31.4	87.9	69.2
CPU (i7-1260P)	ONNX Runtime	4700	10.16 ± 0.92	97.2 ± 2.5	471.8	372.8
FPGA, dataflow (ZCU106)	FINN	187.5	6.87 ± 0	354.8 ± 0	16.1	5.2
FPGA, DPU (ZCU104)	Vitis AI	300	12.17 ± 0.01	120.13 ± 0.03	79.7	11.7

TABLE IV
BENCHMARK ON ARCADE (512×512 INPUT). FLOPS ARE PER IMAGE, BOLD IS BEST PER COLUMN AND SHADED HIGHLIGHTS OUR MODELS.

Model	Params (M)	FLOPs (G)	Mean Dice	LAD	LCX	RCA	LM
nnU-Net (PlainConvUNet) [7]	46.331	134.76	0.8272	0.7018	0.8398	0.9384	0.8288
ResNet101-UNet (pretrained) [18]	62.875	197.40	**0.8474**	**0.7383**	0.8652	**0.9446**	**0.8417**
BSCNet [19]	1.545	3.27	0.7881	0.6874	0.7919	0.8839	0.7891
UNeXt [20]	1.472	4.47	0.7615	0.6220	0.7601	0.8738	0.7902
LightM-UNet [21]	1.343	4.70	0.7239	0.6094	0.7451	0.8984	0.6427
LM-UNet [22]	0.526	7.22	0.8333	0.6781	**0.8983**	0.9341	0.8226
ENet [8]	0.370	4.25	0.8091	0.6861	0.8275	0.9173	0.8055
ERFNet [13]	2.063	29.56	0.8354	0.7344	0.8350	0.9386	0.8337
MobileNetV2 (R-ASPP) [23]	2.109	5.45	0.7775	0.6709	0.7762	0.8745	0.7885
MobileNetV3-Small (LR-ASPP) [24]	0.513	0.72	0.7937	0.6724	0.8059	0.9065	0.7901
SegNet-Basic [25]	1.410	176.45	0.4395	0.3905	0.3453	0.3907	0.6316
Ours-L	0.361	4.21	0.8279	0.7038	0.8520	0.9166	0.8391
Ours-S (FPGA model)	**0.024**	**0.30**	0.7876	0.6523	0.8090	0.9087	0.7804

Table V compares the calibrated and uncalibrated cost models by the absolute percentage error of the estimated resource count relative to the post-synthesis count, $100 \cdot |\text{estimate} - \text{real}|/\text{real}$, over the same four builds (a relative error, not percentage points of device utilization). Calibration reduces the error for every resource and build, most for BRAM and least for DSP, with the remaining LUT and DSP errors being underestimates and the LUT error growing with average bitwidth. Therefore, the resource caps should incorporate a safety margin. While calibration does not eliminate the limitations of the cost model, it should reduce the number of ILP-solve and synthesis iterations needed for hardware deployment.

TABLE V
ABSOLUTE PERCENTAGE ERROR OF THE ESTIMATED RESOURCE COUNT RELATIVE TO THE POST-SYNTHESIS COUNT, OVER THE FOUR BUILDS, BEFORE AND AFTER CALIBRATION.

	Uncalibrated		Calibrated	
	Mean	Max	Mean	Max
LUT	63.6%	77.4%	13.6%	25.6%
BRAM	64.4%	92.1%	5.9%	7.9%
DSP	14.9%	19.8%	9.4%	12.0%

2) Maximum FPS Mode Results: For our proposed dataflow hardware model, we further trained the FP32 version of our optimized network, which increased its mean Dice from

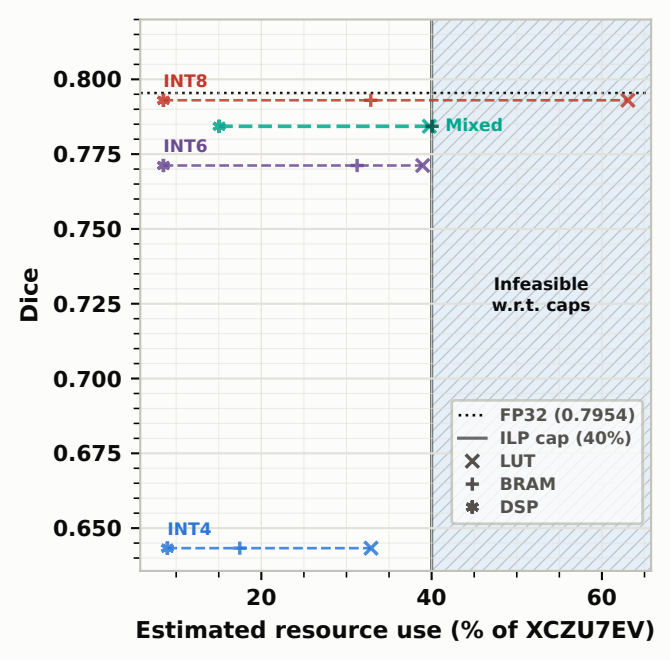


Fig. 5. Dice against estimated LUT, BRAM and DSP use (% of XCZU7EV) of the four builds, solved at a target of 150 FPS and a 40% cap per resource.

0.7876 (Table IV) to 0.7954. The ILP was then solved in the maximum FPS mode, with fixed uniform 8-bit quantization and resource caps of 100%, since the XCZU7EV has sufficient resources for our optimized model.

The deployed design uses 157k LUTs (68%), 45k LUTRAM (44%), 143k flip-flops (31%), 156 BRAMs (50%), 449 DSPs (26%), and no URAM (0%), leaving headroom on the device. It reaches a throughput of 354.8 FPS at a clock of 187.48 MHz, with a mean Dice of 0.7946 after QAT, only 0.08 points below the FP32 model.

3) Comparison Against FINN's Solver: We compared our folding and simulated FIFO sizing (Section IV-B7) against FINN's automatic flow on the uniform 6-bit build of the previous comparison (Fig. 5). FINN can size FIFOs in simulation with either a saturated or a paced source. Our optimizer uses 42% of the device's LUTs, against 70% and 61% for FINN's saturated and paced flows, and 167 DSPs against 270. These LUT and DSP counts are mainly set by the folding and not by the FIFO depths, so they show the advantage of the ILP-based folding. More importantly, ours is the only flow that fits the device for our network. FINN's FIFO sizing requires 5.1–7.6× the device's BRAM capacity, against 33% for ours, owing to our more controlled FIFO sizing. The last partition of FINN's design also fails placement and deadlocks in RTL simulation, whereas ours places and passes simulation. This comes at no throughput cost, as the simulated cycle counts of the three flows differ by less than 0.5%.

C. Hardware Results

To evaluate the performance of our FPGA dataflow accelerator on the ZCU106 development board, we compared it against the DPU overlay on an FPGA, a CPU (i7-1260P) and a GPU (RTX 2050) (Table III). For the DPU flow, we use two B4096 ($8 \times 16 \times 16$) cores on the ZCU104 development board, the largest DPU configuration AMD provides for this board [10], clocked at 300 MHz.

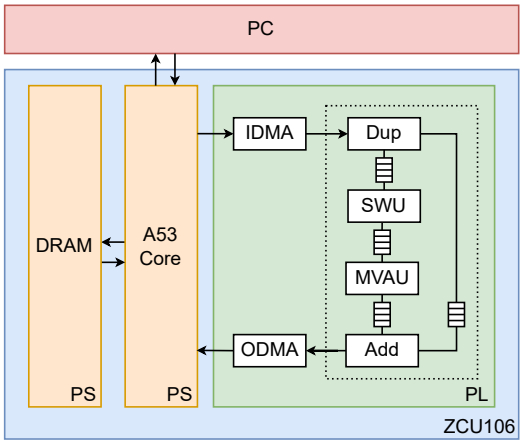


Fig. 6. Abstracted depiction of the deployed accelerator, the hardware system on the ZCU106, and the dataflow. The dotted box shows one residual block as an example, and the white blocks are IPs interconnected by FIFOs.

Figure 6 depicts the platform of our dataflow accelerator on the ZCU106. The processing system (PS) runs a bare-metal application that only transfers images and results between the PC and the accelerator through the input and output direct memory access (DMA) engines (IDMA and ODMA). Otherwise, the accelerator in the programmable logic (PL) is standalone, without any scheduling by the PS.

To make the comparison fair, we deploy our network with uniform 8-bit quantization[3], and measure the inference performance on all platforms[4]. For the GPU, CPU and DPU, we warm up the inference before measurement to exclude kernel setup times.

We measure the Dice, throughput, single-image latency, and energy per image on the ARCADE test split, which is run identically on every platform to obtain the mean and standard deviation.

The single-image latency is the time for one image from submission to result. On the FPGA, it is read from a hardware counter in the PL, from the first input beat to the last output beat of one image, so it includes the filling of the pipeline. The throughput is the streaming throughput, i.e., the sustained rate over a back-to-back stream of the 300 test images at batch size 1, as when frames arrive one at a time from a sensor. In this stream, the pipelined dataflow design processes several images at once, so its throughput is set by its slowest stage (f_{clk}/T, Section IV-B4) rather than by its latency. Therefore, the latency of the FPGA is longer than the inverse of its throughput (6.87 ms against $1/354.8\,\mathrm{FPS} = 2.82\,\mathrm{ms}$, i.e., about 2.4 images in flight), whereas the two are close for the sequential CPU.

[3]Mixed-precision quantization below 8 bits may not be natively supported on all other platforms.

[4]Inference is timed excluding data loading and preprocessing. The DPU stream is processed by one runner per core (two in total), so up to two images are in flight.

All platforms reach a Dice within 0.5 points of each other (0.7901 for the DPU up to 0.7946 for the dataflow accelerator), a difference arising from the different conversion and fine-tuning steps that each platform's software stack requires.

Compared to the GPU, CPU and DPU, our dataflow accelerator consumes 5.0–$29\times$ less total energy per image ($5.5\times$ less than the GPU) and 2.3–$72\times$ less dynamic energy. This is plausibly due to its lower clock frequency (Table III) and its lack of a host operating system and external memory accesses, but we did not isolate these factors experimentally.

We acknowledge the limitations arising from the differing power-measurement boundaries across platforms. The GPU figure covers the GPU board only, excluding the host CPU. The CPU figure covers the processor package, excluding the external memory. The FPGA and DPU figures are sums of the on-board power rails, measured on two different boards (ZCU106 and ZCU104). The total energy per image is the energy over the 300-image stream divided by 300, and the dynamic energy subtracts the idle power of each platform over the same time. The dynamic energy is therefore the closer comparison, particularly for the DPU, whose total is dominated by the idle power of its Linux system ($5.0\times$ total against $2.3\times$ dynamic).

The dataflow accelerator has a deterministic latency, with a spread of ± 0 ms, unlike the GPU and CPU, which show some spread, and the DPU, which is near-deterministic. Its latency is $2.3\times$ higher than the GPU's, but 1.5–$1.8\times$ lower than the CPU's and the DPU's. Notably, it delivers a streaming throughput of 354.8 FPS, which is 1.3–$3.7\times$ that of the other platforms. This is attributed to its pipelined architecture with multiple frames in flight. Batching could raise the throughput of the GPU and CPU, but at the cost of latency, which does not suit the real-time streaming use case targeted here, and we did not evaluate it.

VI. Discussion and Conclusion

Deploying deep-learning models on an FPGA streaming dataflow accelerator presents a large and complex design space. In this work, we proposed an efficient CNN model using a task-aware compression of the ENet model for semantic segmentation of XCA. We further optimized this model for streaming dataflow accelerators, using our proposed ILP optimizer to traverse the trade-off space under hardware resource constraints.

For the CNN architecture, our greedy ablation produced a model with over $14\times$ fewer parameters and FLOPs than the baseline ENet model, at a cost of about 2 Dice points.

As for the optimizer, the resource-aware ILP used fewer LUTs than FINN's solver at essentially equal cycle count, and was the only flow that fit the device and passed RTL simulation. In the maximum accuracy mode, the mixed-precision build reached a higher Dice than the uniform 6-bit build at a similar LUT share, while the uniform 8-bit build exceeded the resource cap. The final model, fine-tuned with QAT, reached a Dice of 0.795 at a streaming throughput of 354.8 FPS with a deterministic latency, using $5.5\times$ less total energy per image than the GPU. This ratio is qualified by the differing power-measurement boundaries of the platforms (Section V-C), and the dynamic energy is the closer comparison.

While the optimizer is demonstrated on one CNN architecture, one segmentation task and one FPGA platform, its broader applicability remains to be validated. Future work includes evaluation on additional network architectures, tasks and FPGA devices, more accurate cost models, and alternative optimization objectives, such as a lexicographic or joint throughput-accuracy objective. It also includes a tuned QAT recipe, support for PReLU, and a common and isolated power-measurement boundary across platforms to support claims of highest energy efficiency for FPGA streaming dataflow architectures. With that said, our ILP-based flow shows potential as a DSE tool for FPGA streaming dataflow architectures, a step toward enabling low-energy and high-throughput inference at the edge.

Acknowledgment

AI tools, including Anthropic Claude, were used for programming and for reviewing the text of this work. All references, design decisions, implementations and results were human-made or human-controlled.

References

[1] Y. Umuroglu, N. J. Fraser *et al.*, "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," *FPGA 2017 - Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 65–74, 12 2016. [Online]. Available: http://arxiv.org/abs/1612.07119http://dx.doi.org/10.1145/3020078.3021744

[2] F. Fahim, B. Hawks *et al.*, "hls4ml: An Open-Source Codesign Workflow to Empower Scientific Low-Power Machine Learning Devices," *TinyML Research Symposium*, vol. 21, 3 2021. [Online]. Available: http://arxiv.org/abs/2103.05579

[3] M. Vaduganathan, G. A. Mensah *et al.*, "The Global Burden of Cardiovascular Diseases and Risk: A Compass for Future Health," *Journal of the American College of Cardiology*, vol. 80, no. 25, pp. 2361–2371, 12 2022. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/36368511/

[4] M. A. Molenaar, E. Hebbo *et al.*, "Deep Learning–Based Segmentation of Coronary Arteries and Stenosis Detection in X-Ray Coronary Angiography," *JACC: Advances*, vol. 4, no. 12P2, p. 102360, 12 2025. [Online]. Available: /doi/pdf/10.1016/j.jacadv.2025.102360?download=truehttps://www.jacc.org/doi/10.1016/j.jacadv.2025.102360

[5] M. Popov, A. Amanturdieva *et al.*, "Dataset for Automatic Region-based Coronary Artery Disease Diagnostics Using X-Ray Angiography Images," *Scientific Data*, vol. 11, no. 1, p. 20, 12 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10764944/

[6] G. Sianos, M.-A. Morel *et al.*, "The SYNTAX Score: an angiographic tool grading the complexity of coronary artery disease," *EuroIntervention : journal of EuroPCR in collaboration with the Working Group on Interventional Cardiology of the European Society of Cardiology*, vol. 1, no. 2, pp. 219–27, 8 2005. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/19758907/

[7] F. Isensee, J. Petersen *et al.*, "nnU-Net: Self-adapting Framework for U-Net-Based Medical Image Segmentation," *Informatik aktuell*, p. 22, 9 2018. [Online]. Available: https://arxiv.org/pdf/1809.10486

[8] A. Paszke, A. Chaurasia *et al.*, "ENet: A Deep Neural Network Architecture for Real-Time Semantic Segmentation," 6 2016. [Online]. Available: https://arxiv.org/pdf/1606.02147

[9] M. Blott, T. B. Preuber *et al.*, "FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 11, no. 3, 9 2018. [Online]. Available: https://arxiv.org/pdf/1809.04570

[10] "DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)." [Online]. Available: https://docs.amd.com/r/en-US/pg338-dpu/

[11] Z. Dong, Z. Yao *et al.*, "HAWQ-V2: Hessian Aware trace-Weighted Quantization of Neural Networks," *Advances in Neural Information Processing Systems*, vol. 2020-December, 11 2019. [Online]. Available: https://arxiv.org/pdf/1911.03852

[12] Z. Yao, Z. Dong *et al.*, "HAWQV3: Dyadic Neural Network Quantization," *Proceedings of Machine Learning Research*, vol. 139, pp. 11 875–11 886, 11 2020. [Online]. Available: https://arxiv.org/pdf/2011.10680

[13] E. Romera, J. M. Alvarez *et al.*, "ERFNet: Efficient Residual Factorized ConvNet for Real-Time Semantic Segmentation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 1, pp. 263–272, 1 2018. [Online]. Available: https://ieeexplore.ieee.org/document/8063438

[14] Z. Chen, J. Gao *et al.*, "FlexCNN: Design of a Universal and Easy-to-Deploy CNN Acceleration System," *Proceedings - IEEE International Symposium on Circuits and Systems*, pp. 534–538, 2026. [Online]. Available: https://ieeexplore.ieee.org/document/11562691

[15] H. Le Blevec, M. Léonardon *et al.*, "FPGA-Oriented Design Space Exploration of a Real-Time Road Scene Semantic Segmentation Deep Neural Network," *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA '25), February 27-March 1, 2025, Monterey, CA, USA*, vol. 1, pp. 54–54, 2 2025. [Online]. Available: /doi/pdf/10.1145/3706628.3708862?download=true

[16] N. Ghielmetti, V. Loncar *et al.*, "Real-time semantic segmentation on FPGAs for autonomous vehicles with hls4ml," *Machine Learning: Science and Technology*, vol. 3, no. 4, 12 2022.

[17] R. Bosio, F. Minnella *et al.*, "NN2FPGA: Optimizing CNN Inference on FPGAs With Binary Integer Programming," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 5, pp. 1807–1818, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10769518

[18] K. He, X. Zhang *et al.*, "Deep Residual Learning for Image Recognition," *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, vol. 2016-December, pp. 770–778, 12 2015. [Online]. Available: https://arxiv.org/pdf/1512.03385

[19] Q. Zhou, L. Wang *et al.*, "Boundary-Guided Lightweight Semantic Segmentation with Multi-Scale Semantic Context," *IEEE Transactions on Multimedia*, vol. 26, pp. 7887–7900, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10462518

[20] J. M. J. Valanarasu and V. M. Patel, "UNeXt: MLP-based Rapid Medical Image Segmentation Network," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 13435 LNCS, pp. 23–33, 3 2022. [Online]. Available: https://arxiv.org/pdf/2203.04967

[21] W. Liao, Y. Zhu *et al.*, "LightM-UNet: Mamba Assists in Lightweight UNet for Medical Image Segmentation," 3 2024. [Online]. Available: https://arxiv.org/pdf/2403.05246

[22] K. Xu, S. Zhou *et al.*, "LM-UNet: Lightweight Mamba-UNet Prostate MRI image segmentation network," *PloS one*, vol. 21, no. 3, 3 2026. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/41871125/

[23] M. Sandler, A. Howard *et al.*, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 1 2018. [Online]. Available: https://arxiv.org/pdf/1801.04381

[24] A. Howard, M. Sandler *et al.*, "Searching for MobileNetV3," *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1314–1324, 5 2019. [Online]. Available: https://arxiv.org/pdf/1905.02244

[25] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 12, pp. 2481–2495, 11 2015. [Online]. Available: https://arxiv.org/pdf/1511.00561